

# 分帳無痛

## 智能群組記帳系統

全端系統期末專案規劃書  
5 天開發完整計畫 | React + Node.js + PostgreSQL

| 技術架構                         | 開發週期    | 預計完成度    |
|------------------------------|---------|----------|
| React + Node.js + PostgreSQL | 5 天全力衝刺 | MVP 完整功能 |

2026 年 6 月

# 目錄

|                                  |    |
|----------------------------------|----|
| 目錄 .....                         | 2  |
| 1. 專案概覽 .....                    | 3  |
| 1.1 系統簡介 .....                   | 3  |
| 1.2 專案目標 .....                   | 3  |
| 1.3 技術選型 .....                   | 3  |
| 2. 系統架構設計 .....                  | 5  |
| 2.1 整體架構 .....                   | 5  |
| 2.2 資料庫設計（6 張資料表） .....          | 5  |
| 2.3 API 端點設計 .....               | 6  |
| 認證模組 /auth .....                 | 6  |
| 群組模組 /groups .....               | 6  |
| 費用模組 /expenses .....             | 6  |
| 結算模組 /settlements .....          | 6  |
| 3. 核心演算法：最優化拆帳 .....             | 8  |
| 3.1 問題定義 .....                   | 8  |
| 3.2 演算法步驟 .....                  | 8  |
| 3.3 演算法範例 .....                  | 8  |
| 4. 五天開發計畫 .....                  | 10 |
| 後端任務（上午） .....                   | 10 |
| 前端任務（下午） .....                   | 10 |
| 群組 API .....                     | 11 |
| 費用 API .....                     | 11 |
| 前端任務 .....                       | 11 |
| 演算法實作（後端 Services） .....         | 12 |
| 結算 API .....                     | 12 |
| 前端結算頁 .....                      | 12 |
| 必做 UI 完整性 .....                  | 13 |
| 加分功能（選做 1-2 項） .....             | 13 |
| 上午：除蟲 & 穩定化 .....                | 14 |
| 下午：部署 .....                      | 14 |
| Demo 腳本（5 分鐘版） .....             | 14 |
| 5. 風險評估與應對策略 .....               | 15 |
| 6. MVP 功能範疇 .....                | 15 |
| 6.1 Must Have（核心功能，必須完成） .....   | 15 |
| 6.2 Should Have（加分功能，盡力完成） ..... | 15 |
| 6.3 Nice to Have（時間允許才做） .....   | 16 |
| 7. 建議專案目錄結構 .....                | 17 |
| 7.1 後端（Node.js） .....            | 17 |
| 7.2 前端（React） .....              | 17 |

# 1. 專案概覽

## 1.1 系統簡介

「分帳無痛」是一套多人群組記帳與智能結算的全端 **Web** 系統。系統核心亮點在於透過貪心演算法計算最少轉帳次數，解決多人分帳時「誰欠誰多少」的複雜計算問題，讓旅遊、合租、聚餐等情境的帳目管理變得輕鬆透明。

### 系統核心價值主張

- 最優化拆帳：**N** 人群組最多只需 **N-1** 次轉帳即可清帳，大幅減少不必要的轉帳往返
- 即時同步：群組成員新增費用後，所有人的餘額立即更新
- 視覺化帳目：圓餅圖顯示各人付款比例，讓帳目一目瞭然
- 多群組管理：同一帳號可同時管理旅遊、合租、聚餐等多個獨立群組

## 1.2 專案目標

| 目標類型 | 具體內容                                             |
|------|--------------------------------------------------|
| 功能目標 | 完成用戶認證、群組管理、費用記帳、最優化結算四大核心模組                     |
| 技術目標 | RESTful API 設計、JWT 驗證、PostgreSQL 資料建模、React 狀態管理 |
| 體驗目標 | 介面操作流暢、支援錯誤提示、空狀態引導、數字精確無誤                       |
| 展示目標 | 系統可部署上線，Demo 流程順暢，能清楚展現演算法的 WOW 效果               |

## 1.3 技術選型

| 層級     | 技術選擇                     | 選用原因            |
|--------|--------------------------|-----------------|
| 前端框架   | React 18 + Vite          | 組件化開發、熱重載、生態豐富  |
| 狀態管理   | Zustand                  | 輕量、簡單、適合中型專案    |
| UI 元件庫 | shadcn/ui + Tailwind CSS | 美觀、易客製、無需額外設定   |
| 圖表     | Recharts                 | React 原生、文件清楚   |
| 後端框架   | Node.js + Express        | JS 全棧一致、開發速度快   |
| 資料庫    | PostgreSQL               | 關聯式資料庫適合帳目計算    |
| 認證     | JWT + bcryptjs           | 無狀態認證、適合 SPA 架構 |

|        |         |                       |
|--------|---------|-----------------------|
| 部署（前端） | Vercel  | 免費、自動化、連接 GitHub 一鍵部署 |
| 部署（後端） | Railway | 免費方案含 PostgreSQL、設定簡單 |

## 2. 系統架構設計

### 2.1 整體架構

系統採用前後端分離架構，前端 React SPA 透過 RESTful API 與後端 Express 伺服器溝通，後端負責業務邏輯與資料庫操作。

#### 架構分層說明

##### 展示層（前端）

- React SPA：Login / Register / 群組首頁 / 費用清單 / 結算頁面
- Zustand 全域狀態：使用者資訊、當前群組、費用列表
- Axios Instance：統一管理 baseURL 與 JWT Authorization Header

##### 業務邏輯層（後端）

- Express Router：依資源分組路由（/auth、/groups、/expenses、/settlements）
- Middleware：authMiddleware 驗證 JWT、錯誤處理中間件
- Services：拆帳演算法、費用計算邏輯獨立封裝

##### 資料層（資料庫）

- PostgreSQL：6 張資料表，Foreign Key 確保資料一致性
- 連接池：使用 pg 套件的 Pool 管理資料庫連線

### 2.2 資料庫設計（6 張資料表）

| 資料表名稱          | 主要欄位                                                                   | 說明                            |
|----------------|------------------------------------------------------------------------|-------------------------------|
| users          | id, name, email, password_hash, created_at                             | 系統使用者基本資訊                     |
| groups         | id, name, description, created_by, created_at                          | 群組基本資訊，created_by 關聯 users.id |
| group_members  | group_id, user_id, joined_at                                           | 多對多關聯表，記錄群組成員                 |
| expenses       | id, group_id, paid_by, amount, description, category, date, created_at | 每筆費用，amount 以「分」為整數單位         |
| expense_splits | id, expense_id, user_id, amount_owed                                   | 費用分攤明細，一筆費用對應多筆分攤記錄           |
| settlements    | id, group_id, from_user_id, to_user_id, amount, settled_at             | 已完成的還款紀錄                      |

### 金額處理重要規則

- 所有金額欄位一律以「分」(integer) 儲存，避免浮點數精度問題
- 例：NT\$350.50 儲存為 35050，顯示時除以 100 並格式化
- 均分計算： $\text{amount} \div N$  取整數，餘數由 `paid_by` 自行承擔（或補給第一位成員）

## 2.3 API 端點設計

### 認證模組 /auth

| Method | 路徑             | 說明                  |
|--------|----------------|---------------------|
| POST   | /auth/register | 新用戶註冊，回傳 JWT token  |
| POST   | /auth/login    | 用戶登入，回傳 JWT token   |
| GET    | /auth/me       | 取得當前登入用戶資訊（需 token） |

### 群組模組 /groups

| Method | 路徑                      | 說明                 |
|--------|-------------------------|--------------------|
| GET    | /groups                 | 取得當前用戶所有群組         |
| POST   | /groups                 | 建立新群組              |
| GET    | /groups/:id             | 取得群組詳情與成員列表        |
| POST   | /groups/:id/members     | 透過 email 加入成員      |
| GET    | /groups/:id/balance     | 計算各成員淨餘額並回傳最優化轉帳清單 |
| GET    | /groups/:id/expenses    | 取得群組所有費用（含付款人資訊）   |
| GET    | /groups/:id/settlements | 取得群組歷史結算紀錄         |

### 費用模組 /expenses

| Method | 路徑                   | 說明                               |
|--------|----------------------|----------------------------------|
| POST   | /groups/:id/expenses | 新增費用（支援均分或自訂金額）                  |
| DELETE | /expenses/:id        | 刪除費用（僅限 <code>paid_by</code> 本人） |

### 結算模組 /settlements

| Method | 路徑                 | 說明         |
|--------|--------------------|------------|
| POST   | /groups/:id/settle | 標記某筆轉帳為已完成 |

### 3. 核心演算法：最優化拆帳

#### 3.1 問題定義

在  $N$  人群組中，每人付款情況各不相同，需計算出讓所有人帳目歸零所需的「最少轉帳次數」。暴力計算的複雜度為指數級，透過貪心演算法可將複雜度降至  $O(N \log N)$ 。

#### 3.2 演算法步驟

##### 貪心演算法 (Greedy Debt Simplification)

1. 計算每人淨餘額：對每位成員，計算  $\text{balance} = \text{已付總額} - \text{應付總額}$
2. 分類： $\text{balance} > 0$  為「債主」（別人欠他）， $\text{balance} < 0$  為「欠款人」（他欠別人）， $\text{balance} = 0$  者排除
3. 排序：債主依  $\text{balance}$  由大到小排序，欠款人依  $|\text{balance}|$  由大到小排序
4. 配對結清：取最大債主  $A$  與最大欠款人  $B$ ，結清較小金額，記錄一筆轉帳  $B \rightarrow A$
5. 更新餘額：調整  $A$  與  $B$  的  $\text{balance}$ ，若任一歸零則從陣列移除
6. 重複步驟 4-5 直到所有人  $\text{balance}$  歸零

最多產生  $N-1$  筆轉帳，遠優於暴力配對可能產生的  $N(N-1)/2$  筆

#### 3.3 演算法範例

情境：小明付了 \$600 餐費，阿花付了 \$300 交通費，大雄付了 \$0，三人均分。

| 成員 | 已付總額  | 淨餘額 (balance)              |
|----|-------|----------------------------|
| 小明 | \$600 | +\$400 (債主)                |
| 阿花 | \$300 | +\$100 (債主)                |
| 大雄 | \$0   | -\$300 (欠款人) (欠均分金額 \$300) |

##### 結算結果

- 步驟 1：最大欠款人大雄 (-\$300) 配最大債主小明 (+\$400)
- 大雄轉帳 \$300 給小明 → 大雄歸零離場，小明剩 +\$100
- 步驟 2：小明 (+\$100) 配阿花 (+\$100) → 無需轉帳，均歸零



最終只需 1 筆轉帳：大雄 → 小明 \$300

## 4. 五天開發計畫

以下為每天的詳細任務分配，每日包含後端 API、前端介面與技術細節。建議搭配 Claude Code 進行開發以加速產出。

**D1**

### 專案環境建置 & 資料庫設計

目標：從零到跑起來，DB Schema 全部定義完、JWT 認證可呼叫

#### 今日產出

- 前後端專案骨架建立完成
- PostgreSQL 資料庫連線成功，6 張資料表建立完畢
- JWT 認證 API (register / login / me) 可正常呼叫
- React 登入 / 註冊頁面串接後端成功

#### 後端任務（上午）

- 初始化 Node.js + Express 專案：npm init -y，安裝 express、pg、jsonwebtoken、bcryptjs、cors、dotenv
- 建立 PostgreSQL 資料庫，執行完整 Schema SQL（含 6 張表與 Foreign Key）
- 實作 POST /auth/register：密碼用 bcrypt hash 後儲存，回傳 JWT
- 實作 POST /auth/login：比對密碼 hash，成功回傳 JWT
- 實作 GET /auth/me：解析 JWT 回傳用戶資訊
- 建立 authMiddleware：驗證 Authorization Header 中的 Bearer token

#### 前端任務（下午）

- 用 Vite 建立 React 專案：npm create vite@latest，選 React + JavaScript
- 安裝相依套件：react-router-dom、axios、zustand、@shadcn/ui、tailwindcss
- 建立 axios instance (src/api/axios.js)：設定 baseURL 與 request interceptor 自動帶入 token
- 建立 Login 頁面與 Register 頁面，表單驗證（必填、email 格式、密碼長度）
- Zustand store 儲存 user 與 token，登入後導向群組首頁

#### Claude Code 建議提示詞

「幫我建立 Node.js + Express 後端專案結構，包含 auth router、authMiddleware、db connection pool，資料庫使用 PostgreSQL，認證使用 JWT + bcrypt」

「幫我生成分帳系統的完整 PostgreSQL Schema，包含 users、groups、group\_members、expenses、expense\_splits、settlements 六張表，含 foreign key 與 index」

## D2

### 群組 & 費用核心 CRUD

目標：可以建群組、加成員、記帳、查看帳目列表

#### 今日產出

- 用戶可建立群組並透過 email 邀請成員加入
- 可新增費用，支援均分與自訂金額兩種分帳方式
- 前端顯示群組列表、費用清單

#### 群組 API

- POST /groups：建立群組，自動將建立者加入 group\_members
- POST /groups/:id/members：透過 email 搜尋用戶後加入群組，用戶不存在回傳 404
- GET /groups：取得當前用戶加入的所有群組（JOIN group\_members）
- GET /groups/:id：群組詳情 + 成員列表（含用戶名稱）

#### 費用 API

- POST /groups/:id/expenses：新增費用，均分時自動計算每人應付金額並插入 expense\_splits
- GET /groups/:id/expenses：費用列表（JOIN users 取得 paid\_by 名稱）
- DELETE /expenses/:id：刪除費用，同步刪除 expense\_splits（僅限 paid\_by 本人）

#### 前端任務

- 建立群組首頁（Dashboard）：顯示用戶所有群組，點擊進入群組詳情
- 新增群組彈窗：輸入群組名稱與描述
- 費用清單頁：顯示每筆費用的金額、付款人、類別、日期
- 新增費用表單：描述、金額、付款人（下拉）、分帳方式切換（均分 / 自訂）、類別選擇

#### 技術注意事項

- 均分餘數處理：amount % N 的餘額讓 paid\_by 自行吸收（最簡單且常見做法）
- 自訂分帳驗證：前端確認各人金額加總等於總費用，後端再次驗證
- 刪除費用要用 Transaction：先刪 expense\_splits 再刪 expenses，避免孤兒資料

## D3

### 最優化拆帳演算法 & 結算功能

目標：實作貪心演算法，這是系統最核心的 WOW 點

#### 今日產出

- 貪心演算法完整實作並通過多種情境測試
- GET /groups/:id/balance 回傳每人淨餘額與最優化轉帳清單
- 前端結算頁顯示轉帳清單，可標記已付款

#### 演算法實作（後端 Services）

- 建立 services/settlement.js，實作 calculateBalances(groupId) 函式
- SQL 查詢：SUM(amount) 按 paid\_by（已付），SUM(amount\_owed) 按 user\_id（應付）
- 計算 balance = paid - owed，過濾 balance = 0 的成員
- 貪心演算法：Max-heap 或排序後雙指針配對，輸出 [{from, to, amount}]
- 撰寫單元測試：3 人、5 人、其中一人已全額付清等情境

#### 結算 API

- GET /groups/:id/balance：呼叫 calculateBalances，回傳 balances 陣列與 settlements 建議清單
- POST /groups/:id/settle：body 帶入 {from\_user\_id, to\_user\_id, amount}，寫入 settlements 表
- GET /groups/:id/settlements：查詢歷史已結算紀錄（JOIN users 取得名稱）

#### 前端結算頁

- 各成員淨餘額卡片：正數顯示綠色（別人欠我），負數顯示紅色（我欠別人）
- 建議轉帳清單：「大雄 → 小明 NT\$300」格式，附「標記已付」按鈕
- 標記已付後即時更新頁面（重新 fetch balance），產生 WOW 效果

#### Claude Code 建議提示詞

「幫我用 JavaScript 實作最優化拆帳演算法：輸入為每個人的淨餘額陣列 [{userId, name, balance}]，使用貪心演算法輸出最少轉帳次數的結算清單 [{from, to, amount}]，balance 為正代表別人欠他，為負代表他欠別人」

## D4

### UI 打磨 & 加分功能

目標：讓介面看起來專業，加入讓評審眼睛一亮的視覺功能

### 今日產出

- 完整響應式 UI，操作流程無障礙
- 費用分類標籤與圖示
- 費用圓餅圖（各成員付款比例）
- 群組儀表板數字卡片（總費用、我欠多少、別人欠我多少）

### 必做 UI 完整性

- Navbar：群組切換下拉選單、用戶頭像 + 登出按鈕
- 群組儀表板卡片：群組總費用、我已付、我尚欠（紅）、別人欠我（綠）
- 費用類別標籤：餐飲、交通、住宿、娛樂、購物、其他，附對應 emoji
- 空狀態畫面設計：無群組 / 無費用時顯示友善引導文案
- Loading 狀態與錯誤提示：API 失敗時不白畫面，顯示清楚錯誤訊息

### 加分功能（選做 1-2 項）

- 費用圓餅圖：Recharts PieChart，顯示各成員付款占比，點擊可查看金額
- 費用篩選：依日期區間、類別篩選費用列表
- 匯出功能：將結算清單匯出為 CSV 或複製為純文字
- 群組邀請碼：產生 6 位隨機邀請碼，他人可透過代碼加入群組

### 時間分配建議

- 60% 時間（約 5 小時）：必做 UI 完整性，確保每個頁面無破版
- 40% 時間（約 3 小時）：加分功能，優先做費用圓餅圖（視覺衝擊最強）
- UI 完美主義是時間殺手，以功能完整性優先

## D5

### 測試、部署 & Demo 準備

目標：系統穩定上線，Demo 流程演練完成，準備讓評審驚艷

### 今日產出

- 系統部署至 Vercel + Railway，可公開訪問
- Demo 資料預植（大阪旅遊群組含 5 筆費用）
- Demo 流程演練完畢，WOW 時刻準備好

上午：除蟲 & 穩定化

- 逐一手動測試所有功能：register → login → 建群組 → 加成員 → 記帳 → 結算 → 標記已付
- 邊界情境測試：群組只有 1 人、金額為 0、刪除所有費用後餘額應回歸 0
- 前端錯誤處理：所有 API 呼叫加入 try/catch，失敗時顯示 toast 通知
- 植入 Demo 資料：建立 3 個預設帳號（小明 / 阿花 / 大雄）與「大阪旅遊」群組，含 5 筆費用

下午：部署

- 後端部署至 Railway：連接 GitHub repo，設定 NODE\_ENV、JWT\_SECRET、DATABASE\_URL
- PostgreSQL 部署：Railway 一鍵開啟 PostgreSQL 外掛，執行 Schema SQL
- 前端部署至 Vercel：連接 GitHub repo，設定 VITE\_API\_URL 環境變數指向 Railway URL
- 驗證跨域：確認 CORS 設定正確，Vercel 域名可正常呼叫 Railway API

Demo 腳本（5 分鐘版）

| 時間   | 操作步驟              | 重點說明                       |
|------|-------------------|----------------------------|
| 0:00 | 開啟系統首頁，展示 UI 整體設計 | 強調乾淨、現代的設計感                |
| 0:30 | 登入「小明」帳號，展示群組列表   | 說明多群組管理概念                  |
| 1:00 | 進入「大阪旅遊」群組，查看費用清單 | 展示費用類別、金額、付款人              |
| 2:00 | 點擊「結算」頁面，展示各人餘額   | 暫停 3 秒讓評審看清楚數字             |
| 2:30 | 展示最優化轉帳清單         | 這是 WOW 時刻！強調「複雜帳目只需 2 筆轉帳」 |
| 3:30 | 點擊「標記已付」，即時更新餘額   | 即時響應展示系統穩定性                |
| 4:00 | 展示費用圓餅圖（如有實作）     | 視覺化亮點                      |
| 4:30 | 總結：說明技術架構與演算法     | 強調貪心演算法的實際應用               |

## 5. 風險評估與應對策略

| 風險項目                   | 發生機率 | 應對策略                                                              |
|------------------------|------|-------------------------------------------------------------------|
| 金額計算出現小數點誤差            | 高    | 所有金額以「分」為單位儲存，顯示時除以 100                                           |
| 拆帳演算法結果不正確             | 中    | Day 3 結束前務必用 3-5 個情境手動驗算                                          |
| 刪除費用後餘額不歸零             | 中    | 刪除費用同時刪除 <code>expense_splits</code> ，使用 <code>Transaction</code> |
| CORS 設定錯誤導致 API 無法呼叫   | 高    | 部署後第一件事驗證 CORS，提前設定好 CORS origin                                  |
| JWT token 過期導致用戶被踢出    | 低    | 設定較長 <code>expiry</code> （7 天），Demo 前重新登入刷新                       |
| Railway 免費方案休眠導致 API 慢 | 中    | Demo 前 5 分鐘先手動呼叫一次 API 喚醒服務                                       |
| 時間不夠，功能沒做完             | 中    | Day 3 演算法是核心，其餘功能優先確保 MVP 穩定                                      |

## 6. MVP 功能範疇

### 6.1 Must Have（核心功能，必須完成）

- 用戶註冊 / 登入 / JWT 認證
- 建立群組 & 邀請成員（透過 email）
- 新增費用（均分）
- 最優化拆帳結算（貪心演算法）
- 標記轉帳已完成

### 6.2 Should Have（加分功能，盡力完成）

- 費用自訂分帳金額
- 費用類別標籤
- 費用圓餅圖視覺化
- 群組儀表板數字摘要卡片

## 6.3 Nice to Have（時間允許才做）

---

- 費用篩選 & 搜尋
- 匯出結算清單（CSV）
- 群組邀請碼功能
- 費用收據拍照辨識



## 7. 建議專案目錄結構

### 7.1 後端 (Node.js)

```
splitwise-backend/
├── src/
│   ├── routes/           # auth.js, groups.js, expenses.js,
│   │   └── settlements.js
│   ├── middleware/       # authMiddleware.js, errorHandler.js
│   ├── services/         # settlement.js (演算法)
│   ├── db/               # pool.js (連線池)
│   └── app.js             # Express 主程式
├── migrations/           # Schema SQL 檔案
├── .env                  # DATABASE_URL, JWT_SECRET, PORT
└── package.json
```

### 7.2 前端 (React)

```
splitwise-frontend/
├── src/
│   ├── pages/            # LoginPage, RegisterPage, DashboardPage,
│   │   ├── GroupPage, SettlePage
│   │   └── components/   # Navbar, ExpenseCard, SettleList, PieChart,
│   │       └── Modal
│   ├── store/            # useAuthStore.js, useGroupStore.js
│   ├── api/              # axios.js (instance + interceptor)
│   ├── utils/            # formatCurrency.js, formatDate.js
│   └── App.jsx            # React Router 路由設定
├── .env                  # VITE_API_URL
└── package.json
```

#### 給 Claude Code 的系統提示詞 (System Prompt)

在 VSCode 中開啟 Claude Code 時，建議使用以下提示詞作為開場，讓 Claude Code 了解整個專案背景：

「我正在開發一個名為『分帳無痛』的全端記帳系統，技術棧為 React + Node.js + PostgreSQL。

系統核心功能：多人群組記帳、JWT 認證、貪心演算法最優化拆帳。

資料庫有 6 張表：users、groups、group\_members、expenses、expense\_splits、settlements。

金額一律以整數（分）儲存。請按照 RESTful API 設計規範，並在每個關鍵函式加上繁體中文注

釋。」